

Symmetric Encryption: How It Works

Contents

Notation (list of symbols)	3
1 Block ciphers vs. stream ciphers	5
1.1 Symmetric encryption in one line	5
1.2 Block ciphers	5
1.3 Stream ciphers	5
1.4 Side by side	6
1.5 Symbols	6
2 Modes of operation	8
2.1 ECB — and why never to use it	8
2.2 CBC — cipher block chaining	8
2.3 CTR — counter mode	9
2.4 Padding the last block	9
2.5 These modes hide, but do not protect	9
2.6 Symbols	10
3 Authenticated encryption	11
3.1 Confidentiality is not integrity	11
3.2 The tool: a message authentication code	11
3.3 Combining a cipher and a MAC — order matters	11
3.4 The AEAD interface	12
3.5 AES-GCM	12
3.6 ChaCha20-Poly1305	13
3.7 Symbols	14
4 Inside ChaCha20	16
4.1 The state: a 4×4 grid of words	16
4.2 The quarter-round	17
4.3 Rounds: columns, then diagonals	17
4.4 Feed-forward, then the keystream	18
4.5 Why ARX	18
4.6 Symbols	19
5 Inside AES	20
5.1 The state and the rounds	20
5.2 SubBytes — the substitution (confusion)	20
5.3 ShiftRows — shuffle across columns (diffusion)	21
5.4 MixColumns — mix within a column (diffusion)	21
5.5 AddRoundKey and the key schedule	21
5.6 SP-network vs. ARX	22
5.7 Symbols	22
6 Nonce management	23
6.1 Why a repeated nonce is a disaster	23
6.2 Two ways to keep nonces unique	23
6.3 The birthday bound on random nonces	23

6.4	Bigger nonces: XChaCha20	24
6.5	Surviving reuse: misuse-resistant AEAD	24
6.6	Which to use	24
6.7	Symbols	25
7	Diffie–Hellman key exchange	26
7.1	The idea: mixing that will not un-mix	26
7.2	The exchange	26
7.3	A worked example	27
7.4	From shared secret to key	27
7.5	The catch: it does not authenticate	27
7.6	In practice: ECDH and forward secrecy	27
7.7	Symbols	28
8	Digital signatures	29
8.1	Key pairs: sign with private, verify with public	29
8.2	Sign the hash, not the message	29
8.3	How it works: RSA and elliptic curves	29
8.4	Certificates: trusting a public key	30
8.5	Authenticating Diffie–Hellman	30
8.6	Symbols	30

Notation (list of symbols)

- k secret key, shared by encryptor and decryptor (symmetric).
- P plaintext: the message to encrypt, a bit string.
- C ciphertext: the encrypted output.
- b block size in bits (e.g. 128 for AES).

1 Block ciphers vs. stream ciphers

A cryptographic hash is *one-way*: you compute a digest and can never recover the message. *Encryption* is the opposite — reversible, with a secret key — and its goal is *confidentiality*: scramble a message so that only a key-holder can read it.

1.1 Symmetric encryption in one line

Symmetric means a single shared secret key k runs both directions:

$$C = \text{Enc}_k(P), \quad P = \text{Dec}_k(C).$$

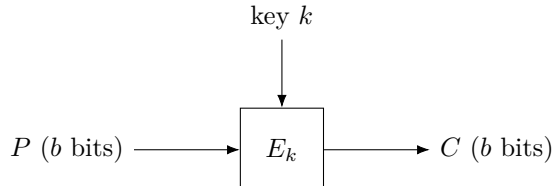
P is the *plaintext* (the message, a bit string), C the *ciphertext* (the scrambled output), and k the key. Dec_k is the exact inverse of Enc_k , so decrypting what you encrypted returns the original. Two families build Enc : *block* ciphers and *stream* ciphers.

1.2 Block ciphers

A block cipher encrypts a *fixed-size block* of bits at a time. Formally it is a keyed *permutation* — a reversible shuffle —

$$E_k : \{0, 1\}^b \rightarrow \{0, 1\}^b,$$

where b is the block size (AES uses $b = 128$ bits). “Permutation” means that for a fixed key, E_k is a bijection on the 2^b possible blocks (one-to-one and onto), so it has an inverse $D_k = E_k^{-1}$ that decryption uses. AES, and the retired DES, are block ciphers.



One block is only b bits, so to encrypt a longer message you need a *mode of operation*: a recipe for chaining many block-cipher calls (CBC, CTR, GCM, ...). Modes usually also require *padding* to fill out the final partial block. (The naive mode, ECB, encrypts each block independently and leaks repeated-block patterns — never use it.)

1.3 Stream ciphers

A stream cipher instead generates a long pseudorandom *keystream* from the key and a *nonce*, and combines it with the plaintext one bit (or byte) at a time with XOR:

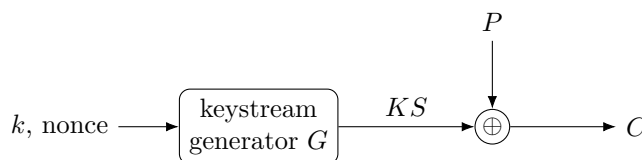
$$KS = G(k, \text{nonce}), \quad C_i = P_i \oplus KS_i, \quad P_i = C_i \oplus KS_i.$$

The jargon:

- $\oplus$ (*XOR*, exclusive-or): a bit operation that outputs 1 exactly when its two input bits differ. Its key property is being self-inverse, $x \oplus y \oplus y = x$ — which is why XORing the ciphertext with the *same* keystream gives the plaintext back.
- *keystream* KS : the pseudorandom bit sequence the cipher produces to mask the message.

- *nonce* (“number used once”, a.k.a. IV): a non-secret value that makes each encryption under the same key produce a *different* keystream. It must never repeat for a given key (see the pitfall below).

This is a practical *one-time pad*: the one-time pad XORs the message with a *truly random* pad as long as the message — perfectly secret but impractical (the key is as big as the data and usable once); a stream cipher replaces that true-random pad with a pseudorandom keystream stretched from a short key. ChaCha20 is the modern choice; RC4 is the well-known broken one.



1.4 Side by side

	Block cipher	Stream cipher
Unit	fixed b -bit block	bit / byte at a time
Mechanism	keyed permutation E_k	XOR with a keystream
Long messages	need a mode (CBC, CTR, GCM)	naturally continuous
Padding	usually required	none
Examples	AES, DES (retired)	ChaCha20, RC4 (broken)
Random access	CTR yes, CBC no	yes

The line blurs. Run a block cipher in *counter* (CTR) mode — encrypt a running counter to make a keystream $KS_i = E_k(\text{nonce} \parallel i)$ and XOR it into the plaintext — and the block cipher *becomes* a stream cipher. So “block vs. stream” is partly about the underlying primitive and partly about how you use it.

The one fatal pitfall: never reuse a (key, nonce) pair. If two messages are XORed with the *same* keystream, then

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

the keystream cancels and the attacker learns $P \oplus P'$ — usually enough to unravel both messages. (This is the “two-time pad” break.) A fresh nonce per message avoids it.

1.5 Symbols

New persistent symbols this section — k, P, C, b — are in the global [list of symbols](#). The symbols below are *local to this section*.

Local symbol	Meaning (this section)
$\text{Enc}_k, \text{Dec}_k$	encrypt / decrypt under key k (exact inverses)
E_k, D_k	block-cipher permutation and its inverse, on b -bit blocks
G	keystream generator
KS	keystream (pseudorandom bits); KS_i is its i -th unit
nonce	“number used once”: non-secret, must not repeat per key
$\oplus$	bitwise XOR
P_i, C_i	i -th bit/byte of plaintext / ciphertext
i	position index

2 Modes of operation

A block cipher E_k encrypts exactly one b -bit block (§1). Real messages are longer, shorter, or not a clean multiple of b . A *mode of operation* is the recipe for calling E_k repeatedly to encrypt a message of any length. Throughout, pad the message (§2.4) and split it into n blocks

$$P_1, P_2, \dots, P_n, \quad \text{each } b \text{ bits,}$$

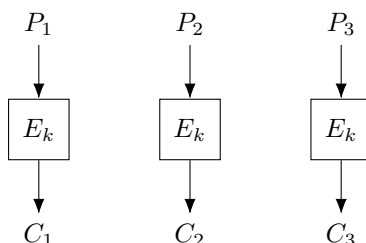
encrypting to ciphertext blocks $C_1, \dots, C_n$. The modes differ in *how* each block is fed through E_k — and that choice decides everything about safety and speed.

2.1 ECB — and why never to use it

The obvious recipe, *Electronic CodeBook* (ECB), encrypts each block on its own:

$$C_i = E_k(P_i), \quad P_i = D_k(C_i),$$

where $D_k = E_k^{-1}$ is the inverse permutation (§1).



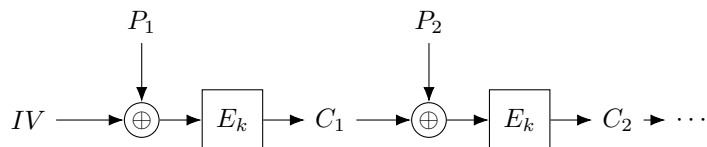
Because E_k is deterministic, *equal plaintext blocks always give equal ciphertext blocks*: if $P_i = P_j$ then $C_i = C_j$. So the ciphertext still shows wherever the message repeats. Encrypt a bitmap image this way and the picture stays recognisable — the infamous “ECB penguin.” ECB leaks structure; never use it for real data. The fix in every mode below is to make encryption *randomised* so identical blocks diverge.

2.2 CBC — cipher block chaining

CBC removes the leak by XORing each plaintext block with the *previous ciphertext block* before encrypting, seeding the chain with a random *initialization vector* IV :

$$C_i = E_k(P_i \oplus C_{i-1}), \quad C_0 \equiv IV.$$

The IV is a non-secret, random b -bit value chosen fresh per message (like the nonce of §1); for CBC it must be *unpredictable*. Because the IV changes, the same plaintext encrypts to different ciphertext each time — the randomisation ECB lacked.



To decrypt, run D_k and undo the XOR. Starting from $C_i = E_k(P_i \oplus C_{i-1})$, apply D_k to both sides: $D_k(C_i) = P_i \oplus C_{i-1}$; then XOR C_{i-1} and use $x \oplus y \oplus y = x$:

$$P_i = D_k(C_i) \oplus C_{i-1}.$$

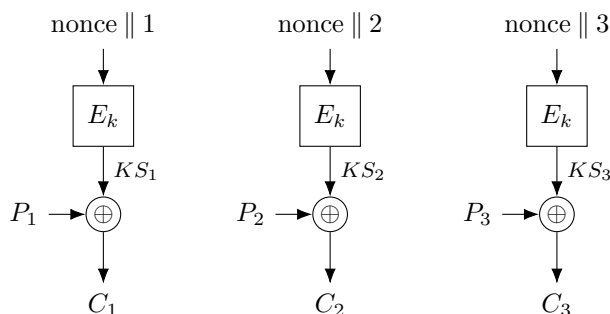
Trade-offs. Encryption is *sequential* — block i needs C_{i-1} , so it cannot be parallelised (decryption can, since all C blocks are already in hand). The message must be padded to a whole number of blocks (below).

2.3 CTR — counter mode

CTR takes the trick from §1: don't encrypt the plaintext at all — encrypt a *counter* to manufacture a keystream, then XOR. For block i ,

$$KS_i = E_k(\text{nonce} \parallel i), \quad C_i = P_i \oplus KS_i, \quad P_i = C_i \oplus KS_i,$$

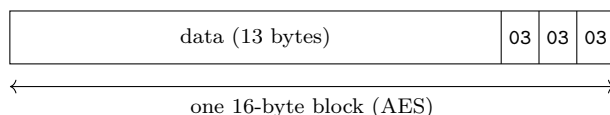
where $\text{nonce} \parallel i$ is the nonce concatenated with the block index i , forming a distinct b -bit input for every block, and KS_i is the resulting keystream block.



Trade-offs. The blocks are independent, so CTR is *fully parallel* and allows *random access* (jump straight to block i). It needs *no padding* (truncate the last keystream block to the message length), and uses only E_k — never the inverse D_k . Encryption and decryption are the *same* operation. The one rule, exactly as in §1: never reuse a (key, nonce) pair, or the keystream repeats and the “two-time pad” break applies.

2.4 Padding the last block

ECB and CBC need the plaintext to fill whole b -bit blocks; CTR and stream ciphers do not. The standard scheme, *PKCS#7*: if p bytes are missing from the final block, append p bytes each holding the *value* p . The value records how many to strip, so removal is unambiguous. If the message already fills the blocks exactly, append one whole extra block of padding — so there is always something to remove.



Here 13 data bytes leave $p = 3$ to fill a 16-byte block, so three 0x03 bytes are appended.

2.5 These modes hide, but do not protect

ECB, CBC, and CTR give *confidentiality* — they scramble the message — but no *integrity*: nothing detects tampering. CTR makes this stark. Since $P_i = C_i \oplus KS_i$, flipping any bit of C_i flips the *same* bit of the recovered plaintext, with no key needed:

$$(C_i \oplus \Delta) \oplus KS_i = P_i \oplus \Delta.$$

An attacker who knows the message format can thus make targeted edits the decryptor cannot notice. Confidentiality is not integrity. Real systems therefore use *authenticated encryption*, which bolts a message authentication code onto a mode (AES-GCM, ChaCha20-Poly1305) so any change is rejected — the subject of its own section.

2.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
E_k, D_k	block-cipher permutation and its inverse, on b -bit blocks
P_i, C_i	i -th b -bit plaintext / ciphertext block
n	number of blocks after padding
IV	initialization vector: random, non-secret, fresh per message
nonce	“number used once”; with i forms CTR’s per-block input
KS_i	i -th keystream block, $E_k(\text{nonce} \parallel i)$ (CTR)
$\oplus$	bitwise XOR
Δ	an attacker’s chosen bit-flip mask
i	block index, $1, \dots, n$

3 Authenticated encryption

The modes of §2 keep a message *secret*, but as §2's last part showed, they do nothing to stop *tampering*: flip a ciphertext bit and the plaintext changes, undetected. Real systems need both. *Authenticated encryption with associated data* (AEAD) is the primitive that delivers it.

3.1 Confidentiality is not integrity

Two distinct goals are in play:

- *confidentiality* — an eavesdropper learns nothing about the message (what §2 gave us);
- *integrity* and *authenticity* — the receiver can be sure the message was not altered, and that it came from someone holding the key.

Encryption alone gives only the first. AEAD gives all of them in one box — and, as a bonus, integrity for some data that travels *in the clear* (packet headers, version numbers).

3.2 The tool: a message authentication code

A *MAC* (message authentication code) is a keyed function that stamps a message with a short *tag*:

$$T = \text{MAC}_{k_m}(M), \quad T \in \{0, 1\}^t \text{ (e.g. } t = 128 \text{ bits)}.$$

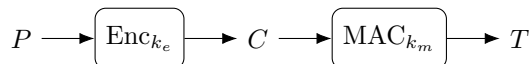
M is the message, k_m the MAC key, and T the tag. To check a received (M, T) , the verifier recomputes $T' = \text{MAC}_{k_m}(M)$ and accepts only if $T' = T$ (compared in *constant time*, so the comparison's duration leaks nothing). Two properties make it useful: only a holder of k_m can produce a valid tag, and changing M by even one bit makes the tag mismatch with overwhelming probability. HMAC (from the hashing collection) is one MAC; GHASH and Poly1305 below are others.

3.3 Combining a cipher and a MAC — order matters

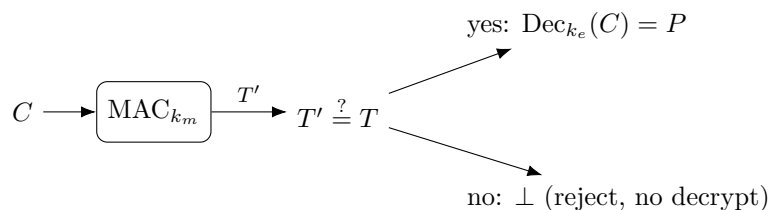
Given a cipher and a MAC, there are three ways to bolt them together — and they are *not* equally safe. Use *independent* keys $k_e \neq k_m$ throughout.

- **Encrypt-and-MAC** (SSH): $C = \text{Enc}_{k_e}(P)$ and $T = \text{MAC}_{k_m}(P)$. The tag is over the *plaintext*, so equal messages give equal tags (a leak), and you must decrypt before you can verify.
- **MAC-then-Encrypt** (old TLS): $T = \text{MAC}_{k_m}(P)$, then $C = \text{Enc}_{k_e}(P \parallel T)$. The receiver must *decrypt* attacker-supplied ciphertext — and strip padding — *before* it can check the tag. That decrypt-first window is exactly what padding-oracle attacks exploited.
- **Encrypt-then-MAC** (IPsec, recommended): $C = \text{Enc}_{k_e}(P)$, then $T = \text{MAC}_{k_m}(C)$. The receiver verifies T against the *ciphertext* first; a bad tag is rejected ($\perp$) *without ever decrypting*. Tampered input never reaches the decryptor.

Encrypt-then-MAC is provably the robust choice, so it is the default.



Encrypt-then-MAC: encrypt P , then MAC the ciphertext; send (C, T) . To verify, the receiver recomputes the tag from C and decrypts only on a match:



3.4 The AEAD interface

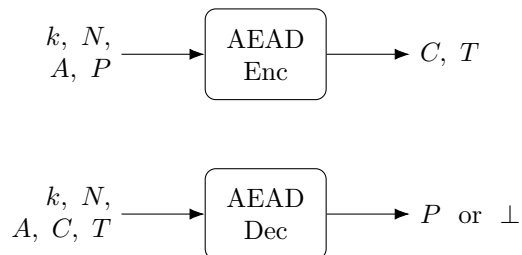
Modern crypto packages a correct composition into a single primitive, so application code cannot pick the wrong order or reuse a key. AEAD has two calls:

$$(C, T) = \text{Enc}_k(N, A, P), \quad \text{Dec}_k(N, A, C, T) \in \{P, \perp\}.$$

The pieces:

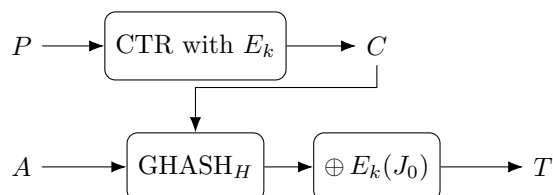
- N — the *nonce*, unique per key (§1);
- A — the *associated data*: authenticated but *not* encrypted, for fields that must stay readable yet tamper-evident (e.g. a packet header);
- T — the tag, which covers *both* C and A ;
- $\perp$ — a distinguished “reject” value returned when the tag does not check out.

Decryption verifies the tag *first* and returns $\perp$ on failure, revealing nothing else — the verify-before-decrypt discipline, now baked in.



3.5 AES-GCM

GCM (Galois/Counter Mode) is Encrypt-then-MAC built from AES. *Confidentiality* comes from CTR mode (§2). *Authenticity* comes from GHASH, a keyed *universal hash* over $\text{GF}(2^{128})$ — arithmetic on 128-bit blocks treated as polynomials, where addition is XOR and multiplication is a carry-less multiply reduced by a fixed polynomial. Its key is $H = E_k(0^{128})$, the cipher applied to the all-zero block.



GHASH folds the blocks $X_1, X_2, \dots$ of $A \parallel C$ (plus a final length block) into a running value, Horner-style, over the field:

$$Y_0 = 0, \quad Y_i = (Y_{i-1} \oplus X_i) \cdot H, \quad T = Y_{\text{last}} \oplus E_k(J_0),$$

where J_0 is the initial counter block derived from the nonce N .

In the terms of §3, the sender transmits $N \parallel A \parallel C \parallel T$: the public nonce N , the associated data A in the clear, the ciphertext C , and the tag T . The receiver, holding k, N, A, C, T , reverses the two lanes in the safe order:

1. *Verify first.* Rebuild $H = E_k(0^{128})$ and J_0 from N , run GHASH over the received A and C , and form $T' = Y_{\text{last}} \oplus E_k(J_0)$. This uses only A and C — never P — so it runs before any decryption. If $T' \neq T$, return $\perp$ and stop.
2. *Then decrypt.* Only on a match, regenerate the CTR keystream from N and XOR it into C to recover P .

That is exactly §3’s verify-before-decrypt: the tag is checked against the ciphertext, so tampered input is rejected before it is ever decrypted.

The final XOR with $E_k(J_0)$ matters: GHASH *alone* is forgeable (a universal hash, not a MAC); masking it with a value that is fresh per nonce is what makes the tag unforgeable.

This is also why **nonce reuse in GCM is doubly fatal** — it breaks CTR confidentiality *and* lets an attacker solve for H , forging tags at will.

GCM is fast where the CPU has AES and carry-less-multiply instructions (AES-NI, PCLMULQDQ), and is standardised as NIST SP 800-38D.

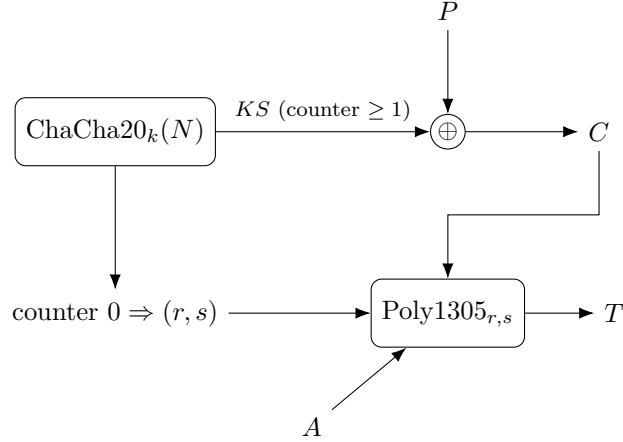
3.6 ChaCha20-Poly1305

The software-friendly alternative pairs the ChaCha20 stream cipher (§1) with the Poly1305 MAC.

First, how ChaCha20 is driven. It does not emit one undifferentiated stream: it produces its keystream in independent 64-byte *blocks*, each picked by a *block counter* — an integer fed in alongside k and N . Evaluating $\text{ChaCha20}_k(N)$ at counter $0, 1, 2, \dots$ hands back successive, unrelated keystream blocks. (So “counter ≥ 1 ” in the diagram means the blocks at counter values $1, 2, \dots$ — it is a block *index*, nothing to do with bit length.)

ChaCha20-Poly1305 puts that to use. It spends the counter-0 block not on the message but on deriving the *one-time* Poly1305 key (r, s) , then encrypts the message with the blocks from counter 1 on. Because the nonce changes per message, (r, s) is fresh every time.

That counter-0 block is 64 bytes, but the key needs only 32: take its *first* 32 bytes (the other 32 are discarded) and split them into two equal halves — r is bytes 0–15, s is bytes 16–31, each 128 bits.



Poly1305 is a polynomial MAC in the prime field modulo $p = 2^{130} - 5$. It folds the 16-byte blocks $c_1, c_2, \dots$ of $A \parallel C$ (plus a length block) into a running value, again Horner-style:

$$a_0 = 0, \quad a_i = (a_{i-1} + c_i) \cdot r \bmod p, \quad T = (a_{\text{last}} + s) \bmod 2^{128}.$$

The key (r, s) must authenticate exactly *one* message — guaranteed by deriving it from the per-message nonce.

Transmission and decryption take the same shape as GCM: send $N \parallel A \parallel C \parallel T$, then on receipt recompute the Poly1305 tag over A and C and compare *before* decrypting, returning $\perp$ on a mismatch.

Poly1305 needs no special hardware and runs in constant time, so it wins on phones and embedded CPUs without AES acceleration. It is standardised as RFC 8439, and used in TLS, WireGuard, and OpenSSH.

	AES-GCM	ChaCha20-Poly1305
Cipher	AES (block, CTR)	ChaCha20 (stream)
MAC	GHASH, $\text{GF}(2^{128})$	Poly1305, mod $2^{130} - 5$
Fastest with	AES-NI hardware	plain software
Nonce reuse	catastrophic	catastrophic
Standard	NIST SP 800-38D	RFC 8439

3.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\text{Enc}_{k_e}, \text{Dec}_{k_e}$	confidentiality-only encrypt / decrypt
MAC_{k_m}	message authentication code; tag $T = \text{MAC}_{k_m}(M)$
k_e, k_m	independent encryption and MAC keys ($k_e \neq k_m$)
N	nonce: unique per key
A	associated data: authenticated, not encrypted
T, T'	tag; and the tag recomputed at verification
$\perp$	“reject”: returned when the tag fails
H	GCM’s GHASH key, $H = E_k(0^{128})$
J_0	GCM’s initial counter block, derived from N
X_i, Y_i	GHASH input blocks and running value
(r, s)	Poly1305 one-time key, from ChaCha20 block 0
c_i, a_i	Poly1305 input blocks and running value

4 Inside ChaCha20

§1 treated the stream cipher’s keystream generator G as a black box, and §3 leaned on ChaCha20’s *block counter* without saying what it is. Now we open the box. ChaCha20 (Bernstein, a refinement of Salsa20) is an *ARX* cipher: it builds its keystream with nothing but **Add**, **Rotate**, and **XOR**.

4.1 The state: a 4×4 grid of words

At heart is a *block function*: given the key, the nonce, and a block counter, it emits one 64-byte keystream block. It works on a state of 16 *words* (1 word = 32 bits), laid out as a 4×4 matrix — $16 \times 32 = 512$ bits = 64 bytes, exactly one block.

The four rows hold four different things:

c_0	c_1	c_2	c_3	4 constants (“expand 32-byte k”)
k_0	k_1	k_2	k_3	256-bit key k (8 words)
k_4	k_5	k_6	k_7	
t	n_0	n_1	n_2	counter t 96-bit nonce (3 words)

The top row is four fixed constants, the same in every ChaCha20 run — and not random-looking magic numbers. They are literally the 16 ASCII bytes of the text **expand 32-byte k** (a 16-character string = 16 bytes = 4 words of 32 bits). Cut the string into four 4-character pieces; each piece, read as a word, is one constant (its bytes packed little-endian, so **expa** becomes 0x61707865):

	c_0	c_1	c_2	c_3
bytes	expa	nd 3	2-by	te k
word	0x61707865	0x3320646e	0x79622d32	0x6b206574

Why text? The constant is there as a fixed value that breaks the state’s symmetry. And choosing it as plain readable text is a *nothing-up-my-sleeve* move: a carefully picked “magic” number could conceal a backdoor, but obvious ASCII — which even spells out that this is the 32-byte-key variant — leaves nowhere to hide one.

The counter t sits in word 12 — this is exactly the block counter of §3. (This is the RFC 8439 layout: a 32-bit counter and a 96-bit nonce.)

Three different time-scales share this one state. The key k is long-lived — reused across many messages. The nonce $n_0n_1n_2$ is drawn *fresh for each message* and held fixed while that message is encrypted. The counter t then runs $0, 1, 2, \dots$ over the 64-byte blocks *within* that single message.

So within one message only t moves: bump it and the whole block changes, giving the next 64 bytes of keystream, while the nonce and key stay put. For the *next* message you draw a new nonce and reset t — the key usually does *not* change, yet the nonce changes anyway. Its freshness is not tied to the key; it must differ every message. That is the (key, nonce)-uniqueness rule of §1 and §3: never let a (key, nonce) pair repeat.

4.2 The quarter-round

All the mixing is done by one primitive, the *quarter-round* QR — a small function of *four* words. A call $\text{QR}(a, b, c, d)$ takes four of the state’s words and updates them in place. The names a, b, c, d are placeholders for “the four words this call happens to work on”; which of the state words $x_0, \dots, x_{15}$ they stand for depends on where QR is applied — fixed by the column/diagonal pattern below. (They are generic argument names, *not* the constants $c_0 \dots c_3$ from above.)

Its four words are updated by these eight steps, run *strictly in sequence* from top to bottom — each step reads the words the steps above it just wrote, so nothing here is parallel:

1. $a \leftarrow a \boxplus b$
2. $d \leftarrow (d \oplus a) \lll 16$
3. $c \leftarrow c \boxplus d$
4. $b \leftarrow (b \oplus c) \lll 12$
5. $a \leftarrow a \boxplus b$
6. $d \leftarrow (d \oplus a) \lll 8$
7. $c \leftarrow c \boxplus d$
8. $b \leftarrow (b \oplus c) \lll 7$

For example, step 2 reads the *new* a that step 1 just wrote, and step 3 the new d from step 2 — the updates chain.

Here $\boxplus$ is addition mod 2^{32} , $\oplus$ is XOR, and $w \lll k$ is a left-rotation of the 32-bit word w by k bits. That is the whole trio: add (mixes across bit positions via carries), rotate (moves bits between positions), XOR (the cheap, reversible combiner).

As a concrete binding: the first column quarter-round (below) sets $a = x_0$, $b = x_4$, $c = x_8$, $d = x_{12}$, while a diagonal one sets $a = x_0$, $b = x_5$, $c = x_{10}$, $d = x_{15}$ — the same recipe, applied to different words.

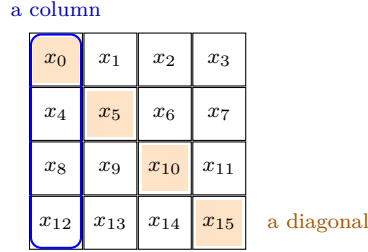
There are no S-boxes and no lookup tables — a deliberate choice we return to in §4 below.

The same quarter-round powers the BLAKE3 hash (in the hashing collection); BLAKE3’s compression function is this mix run with right-rotations and two message words folded in.

4.3 Rounds: columns, then diagonals

One quarter-round touches only four words. To mix all 16, ChaCha alternates between the *columns* and the *diagonals* of the grid. A *double round* is a column round *followed by* a diagonal round (both, in that order — not a choice between them):

column round: $\text{QR}(x_0, x_4, x_8, x_{12}), \text{QR}(x_1, x_5, x_9, x_{13}), \text{QR}(x_2, x_6, x_{10}, x_{14}), \text{QR}(x_3, x_7, x_{11}, x_{15});$
diagonal round: $\text{QR}(x_0, x_5, x_{10}, x_{15}), \text{QR}(x_1, x_6, x_{11}, x_{12}), \text{QR}(x_2, x_7, x_8, x_{13}), \text{QR}(x_3, x_4, x_9, x_{14}).$



The name says the count: **ChaCha20** repeats this fixed pair 10 times — column, diagonal, column, diagonal, ... — for 20 rounds in all. The order never varies, and nothing here is random.

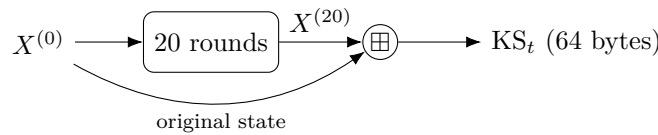
It *must* be deterministic: decryption regenerates the identical keystream from the same key, nonce, and counter (§1), so both sides have to walk exactly the same rounds. The only per-message freshness is the nonce, which is sent in the clear — not a secret random choice.

Diffusion compounds fast — after a couple of double rounds every output word depends on every input word.

4.4 Feed-forward, then the keystream

Call the starting matrix $X^{(0)}$ and the matrix after 20 rounds $X^{(20)}$. The block function does not output $X^{(20)}$ directly; it first adds the *original* state back in, word by word (mod 2^{32}):

$$\text{KS}_t = X^{(20)} \boxplus X^{(0)}.$$



That feed-forward is what makes the block function one-way: the 20 rounds alone are a reversible permutation, but adding $X^{(0)}$ back means you cannot run it backwards to recover the key or counter from the keystream — the same trick as SHA-2’s Davies–Meyer (in the hashing collection).

The result KS_t is the 64-byte keystream block for counter t . To encrypt, do what §1 and §2 described: generate $\text{KS}_0, \text{KS}_1, \dots$ by incrementing t , and XOR them onto the plaintext.

This is the missing piece from §3: in ChaCha20-Poly1305 the counter-0 block supplies the Poly1305 key (its first 32 bytes) and the message rides on counters $1, 2, \dots$

4.5 Why ARX

The all-ARX design buys two things. Because every step is arithmetic on whole words — never a table indexed by secret data — ChaCha20 runs in *constant time*, immune to the cache-timing leaks that dog naive S-box ciphers.

And ARX is cheap and simple on any CPU, with no special hardware needed — which is precisely why ChaCha20-Poly1305 (§3) is the pick on phones and other machines without AES acceleration.

The 20 rounds are a generous margin: the best known attacks reach only about 7.

4.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
word	32 bits
$x_0, \dots, x_{15}$	the 16-word (4×4) ChaCha20 state
$c_0, \dots, c_3$	the four fixed constants (“ expand 32-byte k ”)
$k_0, \dots, k_7$	the eight words of the 256-bit key k
t	block counter (state word 12)
n_0, n_1, n_2	the three words of the 96-bit nonce
QR	the quarter-round (mixes four state words)
$\boxplus$	addition mod 2^{32} (word-wise)
$\lll k$	left-rotation of a 32-bit word by k bits
$X^{(0)}, X^{(20)}$	the state before / after the 20 rounds
KS_t	the 64-byte keystream block for counter t

5 Inside AES

§4 built a cipher from pure ARX. AES (Rijndael, the NIST standard since 2001) takes the other classic route: a *substitution-permutation network* (SP-network) — alternating a byte-substitution layer with a byte-shuffling layer, interleaved with key XORs.

Unlike ChaCha20, AES is a *block cipher* (§1): it maps one 128-bit block to one 128-bit block under the key, and to encrypt a real message you wrap it in a mode (§2).

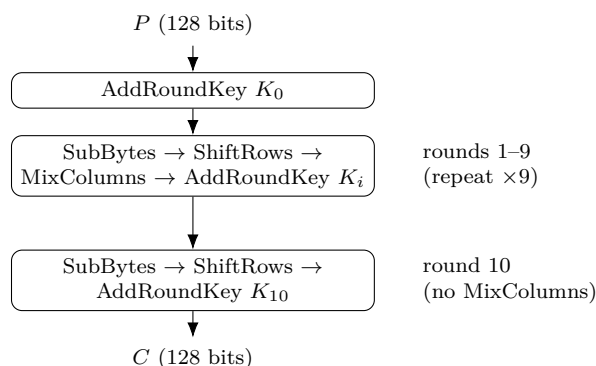
5.1 The state and the rounds

AES works on a *state*: the 128-bit block written as a 4×4 grid of *bytes* (16 bytes = 128 bits). Write $s_{r,c}$ for the byte in row r , column c ($r, c \in \{0, 1, 2, 3\}$). The input bytes fill the grid down columns — bytes 0–3 are column 0, bytes 4–7 column 1, and so on.

$s_{0,0}$	$s_{0,1}$	$s_{0,2}$	$s_{0,3}$
$s_{1,0}$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$
$s_{2,0}$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$
$s_{3,0}$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$

16 bytes, 4×4 (= 128 bits)

A 128-bit key gives $N_r = 10$ rounds (a 192-bit key gives 12, a 256-bit key 14). Each full round is four steps in fixed order — **SubBytes**, **ShiftRows**, **MixColumns**, **AddRoundKey** — with one twist at the ends:



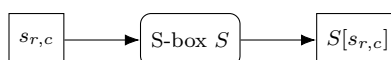
An extra AddRoundKey runs *before* round 1 (so no plaintext block is ever sent through the rounds unkeyed), and the *final* round drops MixColumns.

Three of the four steps — SubBytes, ShiftRows, MixColumns — are fixed, public, key-free shuffles. Only AddRoundKey injects the secret. The security comes from repeating the substitute-shuffle-mix-key alternation enough times that the two are hopelessly entangled.

5.2 SubBytes — the substitution (confusion)

Each byte is replaced by another via a fixed 256-entry lookup table, the *S-box* S :

$$s_{r,c} \leftarrow S[s_{r,c}] \quad \text{for all } r, c.$$



This is the cipher’s *only* nonlinear step, and it supplies *confusion* (Shannon’s term): it makes the relationship between key and ciphertext intricate, so the output cannot be unpicked with linear algebra. The S-box itself is built from the multiplicative inverse in $\text{GF}(2^8)$ followed by a fixed affine map — chosen for strong nonlinearity — where $\text{GF}(2^8)$ is byte arithmetic (each byte a polynomial; add = XOR, multiply = carry-less multiply reduced by a fixed degree-8 polynomial).

5.3 ShiftRows — shuffle across columns (diffusion)

Row r is cyclically shifted left by r bytes: row 0 stays put, row 1 moves one place, row 2 two, row 3 three. (Cells show the byte index, numbered down columns.)

0	4	8	12		0	4	8	12	
1	5	9	13		5	9	13	1	shift 1
2	6	10	14		10	14	2	6	shift 2
3	7	11	15		15	3	7	11	shift 3
before					after				

On its own this just moves bytes around. Its job is to feed MixColumns: after the shift, each column holds bytes that started in four *different* columns, so the next step mixes bytes from all over the block — the start of *diffusion* (spreading each input’s influence widely).

5.4 MixColumns — mix within a column (diffusion)

Each column is treated as a 4-byte vector and multiplied by a fixed matrix over $\text{GF}(2^8)$:

$$\begin{pmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{pmatrix} = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{pmatrix} \quad (c = 0, \dots, 3).$$

Spelling out the first row, with $\cdot$ a $\text{GF}(2^8)$ multiply and $\oplus$ XOR:

$$s'_{0,c} = (2 \cdot s_{0,c}) \oplus (3 \cdot s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}.$$

The constants 1, 2, 3 are just small field elements. ShiftRows and MixColumns together give full diffusion: after about two rounds every output byte depends on every input byte. (The final round omits MixColumns — at the very end it adds no security, and leaving it out keeps encryption and decryption structurally aligned.)

5.5 AddRoundKey and the key schedule

AddRoundKey XORs the round key K_i (also 128 bits, a 4×4 byte grid) into the state, byte for byte:

$$s_{r,c} \leftarrow s_{r,c} \oplus (K_i)_{r,c}.$$

This is the only step that touches the secret. XOR is its own inverse, so decryption removes the same key with another XOR.

The $N_r + 1 = 11$ round keys are not 11 independent secrets — they are stretched from the one key by the *key schedule*. Take the key as four 4-byte words $W_0, \dots, W_3$; the rest follow a recurrence. For

most words $W_i = W_{i-4} \oplus W_{i-1}$; but every fourth word first passes W_{i-1} through a small function g (rotate its bytes, push them through the S-box, then XOR a round constant Rcon):



Round key K_i is the four words $W_{4i}, \dots, W_{4i+3}$. Feeding the words through the S-box and the changing Rcon makes the round keys differ unpredictably, so learning one tells an attacker little about the others.

5.6 SP-network vs. ARX

AES and ChaCha20 (§4) reach the same goal by opposite means: AES leans on a lookup-table S-box plus linear mixing, ChaCha20 on add/rotate/XOR with no tables at all.

That S-box is the catch. Implemented naively as a table indexed by secret bytes, it can leak through cache-timing; this is exactly the weakness ChaCha20's table-free design avoids. AES's answer is hardware: the **AES-NI** instructions do a whole round at once, making AES both very fast *and* constant-time on modern CPUs.

That split is why §3's two AEADs coexist: AES-GCM wins where AES-NI is present, ChaCha20-Poly1305 where it is not.

	AES	ChaCha20
Type	block cipher	stream cipher
Design	SP-network (S-box)	ARX (no tables)
Unit	byte ($\text{GF}(2^8)$)	32-bit word
Nonlinearity	the S-box	carry in $\boxplus$
Fastest with	AES-NI hardware	plain software

5.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$s_{r,c}$	state byte at row r , column c ; the state is 4×4 bytes = 128 bits
N_r	number of rounds (10 for AES-128)
$S[\cdot]$	the S-box: a fixed 256-entry byte substitution
K_i	the i -th round key (128 bits), $i = 0, \dots, N_r$
$\text{GF}(2^8)$	byte field: add = XOR, multiply = carry-less mod a fixed polynomial
W_i	key-schedule words (4 bytes each); $K_i = (W_{4i}, \dots, W_{4i+3})$
RotWord, SubWord, Rcon	key-schedule helpers (rotate, S-box, round constant)
$\oplus$	bitwise XOR

6 Nonce management

§1, §2, and §3 all ended with the same warning: never reuse a (key, nonce) pair. This section is about how to actually *guarantee* that — and what to do when you cannot.

6.1 Why a repeated nonce is a disaster

Reuse breaks things in two separate ways.

Confidentiality. A repeated (key, nonce) gives the *same* keystream KS . Two messages masked with it satisfy

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

so the keystream cancels and the attacker learns $P \oplus P'$ — the two-time-pad break of §1.

Authenticity. In GCM (§3) a repeat is worse still: it lets the attacker solve for the GHASH key H and then *forge* tags for messages of their choosing. So a single slip turns into a total break, not just a leak.

That is why nonce uniqueness is load-bearing for the whole construction — and why it deserves real care.

6.2 Two ways to keep nonces unique

There are two strategies, with opposite failure modes.

Counter nonces (stateful): hold a counter and increment it for every message, never repeating a value. This gives the largest message budget — a 96-bit counter never wraps in practice. The danger is the *state*: a reboot, a restored backup, or a cloned VM can rewind the counter and silently reissue a value.

Random nonces (stateless): draw a fresh random nonce each time. There is no state to lose. The danger is *chance*: two messages may happen to draw the same nonce — the birthday problem, next.

6.3 The birthday bound on random nonces

With n -bit random nonces, after q messages under one key the chance that two coincide is about

$$\Pr[\text{collision}] \approx \frac{q^2}{2^{n+1}}.$$

To keep that negligible (say below 2^{-32}), you need $q \lesssim 2^{(n-32)/2}$. That ceiling is lower than people expect:

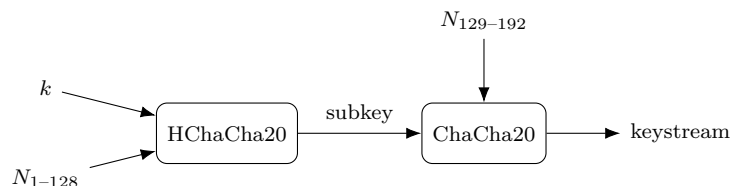
Nonce size	Used by	Random nonces safe up to
96 bits	GCM, ChaCha20 (§3)	$\approx 2^{32}$ messages
192 bits	XChaCha20 (below)	$\approx 2^{80}$ messages

So a 96-bit random nonce is fine for billions of messages but not for an unbounded stream; past the limit you must rekey — or use a bigger nonce.

6.4 Bigger nonces: XChaCha20

The clean fix for random nonces is to make the nonce so large that collisions never happen. XChaCha20 extends ChaCha20’s 96-bit nonce to 192 bits.

It does so in two stages. First it runs *HChaCha20* — the ChaCha core (§4) used as a key-derivation step — on the key and the *first* 128 bits of the nonce, giving a fresh 256-bit subkey. Then it runs ordinary ChaCha20 under that subkey with the remaining 64 nonce bits.

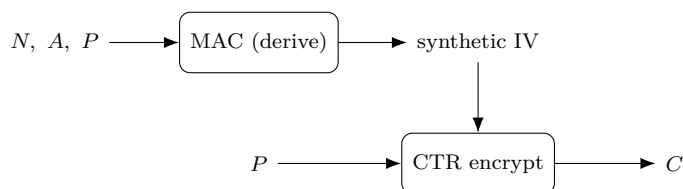


The upshot is a 192-bit nonce, whose birthday bound ($\approx 2^{80}$ messages) is so far off that you can pick nonces at random and never think about collisions again. (XSalsa20 does the same for Salsa20.)

6.5 Surviving reuse: misuse-resistant AEAD

A different philosophy: rather than *prevent* reuse, limit the damage when it happens. That is what AES-GCM-SIV does (SIV = *synthetic IV*).

Its trick is to derive the encryption IV from a MAC of the nonce, the associated data, *and the plaintext*, then CTR-encrypt with that IV (which also serves as the tag).



Because the IV depends on the message content, two *different* messages under the same (key, nonce) get different IVs — hence different keystreams, and no two-time pad. The only thing a repeat leaks is whether the *exact same* message was sent twice (identical ciphertext). That is a mild, bounded leak next to GCM’s catastrophe.

The cost: the IV must be computed from the whole plaintext before encryption can start, so AES-GCM-SIV needs two passes and cannot encrypt a stream on the fly. You trade a little speed for robustness against a footgun.

6.6 Which to use

Situation	Use
Reliable counter state	a counter nonce (largest budget)
Stateless or many senders	random nonces — prefer 192-bit (XChaCha20)
Uniqueness can’t be guaranteed	a misuse-resistant AEAD (AES-GCM-SIV)

Underneath all three, the rule never changes: a (key, nonce) pair must not repeat. When you are unsure whether it might, rekey.

6.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
N	the nonce
n	nonce length in bits
q	number of messages encrypted under one key
KS	keystream (§1)
subkey	the 256-bit ChaCha20 key HChaCha20 derives (XChaCha20)
synthetic IV	an IV derived from the message itself (AES-GCM-SIV)

7 Diffie–Hellman key exchange

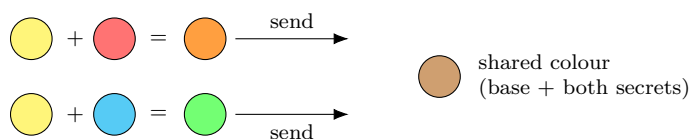
Every section so far opened with “a shared secret key k ” and simply assumed both sides had it. But how do two parties who have never met agree on k over a channel an eavesdropper can read? You cannot just send k — the eavesdropper would see it.

Diffie–Hellman (1976) solves exactly this. It is the first piece of *public-key* crypto in this document: a way for two parties to agree on a shared secret in the open, where the eavesdropper sees *every* message exchanged and still cannot compute the secret.

7.1 The idea: mixing that will not un-mix

The trick is an operation that is easy to do but hard to undo. The classic intuition is *paint*: mixing two colours is easy, but separating a mixture back into its ingredients is practically impossible.

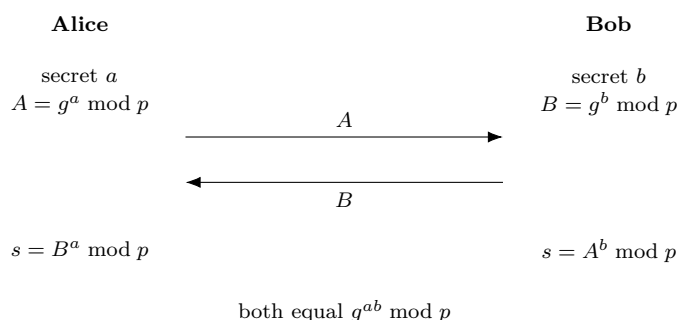
Both parties start from a public base colour. Each keeps a secret colour and mixes it into the base; they swap the resulting mixtures in the open. Each then stirs in their own secret again — and both arrive at the *same* final colour (base + both secrets), while an eavesdropper, holding only the two public mixtures, cannot un-mix them to reach it.



The mathematical “mixing” is *modular exponentiation*. Fix a large public prime p and a public base g . Raising g to a power mod p is easy; but recovering the exponent from the result — the *discrete logarithm* — is believed infeasible for large p . That one-wayness is what the eavesdropper runs into.

7.2 The exchange

Public, known to everyone: the prime p and the base g . Each party draws one private exponent.



Both land on the same value because the exponents just multiply:

$$B^a = (g^b)^a = g^{ab} = (g^a)^b = A^b \pmod{p}.$$

The eavesdropper sees g , p , $A = g^a$, and $B = g^b$, but to get g^{ab} they would need a or b — i.e. solve a discrete logarithm. That is the *Diffie–Hellman problem*, believed hard for well-chosen large p (today, 2048 bits or more).

Notation flag. The modulus here is the lower-case prime p , *not* the plaintext P used elsewhere in this document.

7.3 A worked example

Small numbers make it concrete. Take public $p = 23$, $g = 5$.

- Alice picks $a = 6$ and sends $A = 5^6 \bmod 23 = 8$.
- Bob picks $b = 15$ and sends $B = 5^{15} \bmod 23 = 19$.
- Alice computes $s = B^a = 19^6 \bmod 23 = 2$.
- Bob computes $s = A^b = 8^{15} \bmod 23 = 2$.

Both get $s = 2$, which equals $g^{ab} = 5^{90} \bmod 23 = 2$. An eavesdropper holds $g = 5$, $p = 23$, $A = 8$, $B = 19$ — and to find a must solve $5^a \equiv 8 \pmod{23}$ by discrete log. At 23 that is a quick search; at 2048 bits it is hopeless.

7.4 From shared secret to key

The shared s is a big number, not yet a key. You do not use it directly: run it through a key-derivation function (a hash-based KDF, from the hashing collection) to get a uniform symmetric key

$$k = \text{KDF}(s).$$

From there, everything in §1–§6 applies — k feeds AES-GCM or ChaCha20-Poly1305 as usual. Diffie–Hellman is the missing first step that produces the k the rest of the document assumed.

7.5 The catch: it does not authenticate

Plain Diffie–Hellman defeats a *passive* eavesdropper, but not an *active* attacker. Nothing in the exchange proves *who* is on the other end.

A *man-in-the-middle*, Mallory, can run a separate exchange with each side — agreeing on a secret s_1 with Alice and a different s_2 with Bob — then sit in the middle, decrypting with one key and re-encrypting with the other, reading everything. Neither side notices.



So the exchange must be *authenticated*: the public values are tied to an identity by a signature, a certificate, or a pre-shared key. TLS, for instance, has the server sign its Diffie–Hellman value. Diffie–Hellman gives secrecy of the agreement; proving who you agreed *with* is a separate, mandatory job.

7.6 In practice: ECDH and forward secrecy

Two refinements dominate real use.

Elliptic-curve Diffie–Hellman (ECDH) runs the same idea in a different group: points on an elliptic curve, where “raise to a power” becomes “multiply a point by a scalar.” The hard problem is the same shape, but breaking it needs far bigger effort per bit — so a 256-bit curve key (e.g. X25519) matches a 3072-bit classical one. Smaller and faster, it is the modern default.

Forward secrecy comes from using *ephemeral* secrets — fresh a, b per session, discarded afterwards (the “E” in ECDHE). Then even if a party’s long-term key is later stolen, past sessions stay secret: their one-time exponents are already gone.

7.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
p	large public prime, the modulus (<i>not</i> the plaintext P)
g	public base (generator)
a, b	Alice’s and Bob’s private exponents (secret)
A, B	public values $g^a \bmod p$ and $g^b \bmod p$
s	the shared secret $g^{ab} \bmod p$
KDF	key-derivation function turning s into the key k

8 Digital signatures

§7 left a hole: Diffie–Hellman agrees a key but cannot prove *who* is on the other end, so a man-in-the-middle slips in. Closing that hole is what digital signatures do.

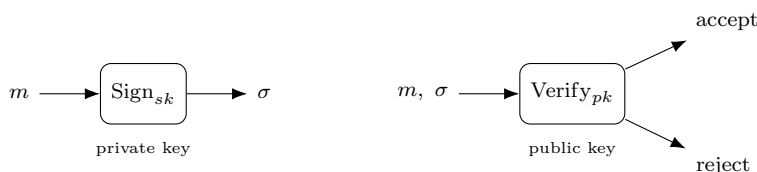
Could a MAC (§3) prove identity? No — a MAC needs a *shared* secret, and a shared secret is exactly what we have not got yet. We need to prove a message came from a specific party with *no* pre-shared secret, in a way *anyone* can check. That is a signature.

8.1 Key pairs: sign with private, verify with public

Each party holds a *key pair*: a *private key* sk kept secret, and a *public key* pk published to the world. They drive two operations:

$$\sigma = \text{Sign}_{sk}(m), \quad \text{Verify}_{pk}(m, \sigma) \in \{\text{accept}, \text{reject}\}.$$

Here m is the message and σ its signature.



Only the holder of sk can produce a σ that verifies, and pk reveals nothing that helps forge one. So *anyone* can verify, but only *one* party can sign.

That asymmetry is the whole difference from a MAC:

	MAC (§3)	Signature
Keys	one shared secret	private (sign) + public (verify)
Who can sign	both parties	only the private-key holder
Who can verify	both parties	anyone with pk
Shared secret	required	none
Non-repudiation	no	yes

The last row matters: because only the signer holds sk , they cannot later deny a valid signature — *non-repudiation*, something a MAC (where both sides share the key) can never give.

8.2 Sign the hash, not the message

The signature math takes a fixed-size input, but messages are any length. So you hash first (*hash-then-sign*):

$$\sigma = \text{Sign}_{sk}(H(m)),$$

where H is a collision-resistant hash (from the hashing collection). Collision resistance is essential: if $H(m) = H(m')$, one signature would be valid for both messages.

8.3 How it works: RSA and elliptic curves

Every scheme rests on a *trapdoor*: signing is easy with sk , forging is infeasible without it.

RSA (trapdoor: factoring is hard). The public key is a modulus $N = pq$ (product of two big primes) and an exponent e ; the private key is d with $ed \equiv 1 \pmod{\varphi(N)}$. Signing applies the private exponent, verifying the public one, and they undo each other:

$$\sigma = H(m)^d \pmod{N}, \quad \text{accept iff } \sigma^e \equiv H(m) \pmod{N}.$$

Forging needs d , which needs factoring N — infeasible for large N . (Real RSA pads the hash first, e.g. PSS; signing the bare hash is unsafe.)

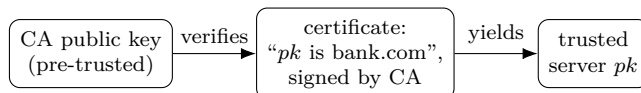
Elliptic curves (trapdoor: the curve discrete log of §7). The private key is a secret scalar, the public key a curve point. ECDSA and Ed25519 are the modern schemes — much smaller and faster than RSA for equal security.

A nonce footgun, again. ECDSA needs a fresh, unpredictable secret value for *each* signature. Reuse it — or let it be guessable — across two signatures and the private key falls straight out, the same nonce-reuse disaster as §6. It has broken real systems (the Sony PS3, early Bitcoin wallets). Ed25519 removes the footgun by deriving that value deterministically from the key and message.

8.4 Certificates: trusting a public key

A signature proves “whoever holds sk signed this” — but how do you know a given pk belongs to the *right* party (the real bank, not Mallory)?

A *certificate* answers that: a trusted *Certificate Authority* (CA) signs a statement binding a public key to an identity. You check that certificate with the CA’s *own* public key, which you already trust because it ships with your OS or browser.



This is the *chain of trust*: trust in one pre-installed CA key extends, via signatures, to the public key of any site it vouches for.

8.5 Authenticating Diffie–Hellman

Now §7’s man-in-the-middle is finished. Each party *signs* its Diffie–Hellman public value with its private key; the other verifies the signature with the partner’s certificate-vouched public key.

Mallory cannot forge that signature, so she cannot substitute her own Diffie–Hellman value without being caught — the exchange is now *authenticated*. Key agreement (§7) plus signed identities is the combination every real secure channel is built from.

8.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
sk, pk	private (signing) key and public (verification) key
Sign, Verify	the signing and verification operations
σ	the signature
$m, H(m)$	the message and its hash (what actually gets signed)
N, e, d	RSA modulus $N = pq$, public exponent e , private exponent d
CA	Certificate Authority: vouches for a public key